

SLM-Based CVE Triage

Summer Li, Tara Dixit, Jessie Dong

Emails: summerli@stanford.edu, tdixit@stanford.edu, jdong27@stanford.edu

1 Introduction

When a software vulnerability is publicly disclosed, it is usually assigned a CVE identifier, which gives security teams a standard way to refer to that specific flaw. A CVE record includes a short description of the vulnerability, but the description alone is not enough to decide how urgent the issue is. Security teams also need structured information such as a CVSS score from 0 to 10, a severity label derived from that score (CRITICAL, HIGH, MEDIUM, or LOW), a CWE category for the type of weakness, and a CVSS vector string encoding specific attack characteristics such as delivery mechanism and required privileges. Scanners, patching tools, and incident response teams use this information to decide what to fix first.

Today, this enrichment is produced by the National Vulnerability Database, or NVD. However, the NVD has struggled to keep up with the growing number of disclosed vulnerabilities. In 2025, 131 new CVEs were disclosed each day, and in April 2026, NIST announced that it would stop assigning severity scores. This creates a gap between when a vulnerability is disclosed and when defenders have the information needed to prioritize it.

This project studies whether small language models can help automate CVE enrichment. Given only a raw CVE description, the model predicts the CVSS base score, severity bucket, CWE class, and eight CVSS vector components. We fine-tune Phi-3.5-mini, a 3.8B parameter model, using LoRA, and compare it against TF-IDF baselines with logistic regression and random forest. We also test a smaller Qwen 0.5B model as a lower-bound comparison. The goal is to evaluate whether a lightweight model can produce useful CVE triage information from public data while remaining practical to train on consumer hardware.

The input to our system is a raw CVE description string. We use TF-IDF with logistic regression, TF-IDF with random forest, and LoRA fine-tuned Phi-3.5-mini to jointly output a CVSS base score (regression), severity label (four-class), CWE category (~150-class), and eight CVSS vector components (categorical).

2 Related Work

Classical ML. Aota et al. showed that TF-IDF models can perform well on vulnerability classification, reaching 74% accuracy and even outperforming some LLM-based approaches. These methods are fast, cheap, and easy to interpret. However, they treat descriptions as a collection of keywords, so they can miss meanings that depend on context, such as whether a vulnerability can be exploited remotely, requires user interaction, or only applies under specific conditions (2).

Transformer-based classification. Other work uses transformer models to analyze vulnerability descriptions. CVSS-BERT, which stands for Bidirectional Encoder Representations from Transformers, is a model that predicts some of the eight additional factors behind the CVSS score (3). The Computer Incident Response Center Luxembourg (CIRCL) has released a RoBERTa model for vulnerability severity and CWE classification (4). A 2026 arXiv preprint found that a fine-tuned RoBERTa-125M model can perform competitively on CVE-to-CWE mapping (5). These models capture more context than TF-IDF, but are mostly designed for a single prediction task, such as severity-only or CWE-only, rather than producing all enrichment fields together.

LLM-based approaches. More recent work has explored whether large language models can help with vulnerability triage. CVE-LLM is a German company Siemens Healthineers system for evaluating medical-device vulnerabilities (6). Additionally, CASEY is a research system that uses GPT models to predict a vulnerability’s CWE category and severity (7). A Exploit Prediction Scoring System (EPSS) predicts how likely a CVE is to be exploited in the wild, and its team found that zero-shot LLMs were not as strong as their statistical model (8). Databricks, a data and AI company, built VulnWatch to help prioritize vulnerabilities, but it also depends on closed cloud infrastructure (10). These examples show that LLMs can be useful for this task, but many existing approaches are either too narrow or require proprietary technology.

Where our work fits. All prior attempts usually have at least one major limitation: they predict only one field at a time, rely on proprietary models, or require cloud infrastructure. Our work predicts all four outputs in a single model pass, runs locally on a 16 GB MacBook, and is fully open-source.

3 Dataset and Features

Source. All data comes from the National Vulnerability Database (NVD). Instead of downloading each CVE one at a time from the NVD API, we use the Fraunhofer FKIE community mirror, which is a public copy of the NVD data maintained by German applied research institute Fraunhofer FKIE. This mirror organizes the NVD JSON feeds into yearly files and updates them every two hours, making the dataset easier for us to download and process (11).

Splits. We split the data by year. We use CVEs from 2022 and 2023 for training and validation. This gives us 50,424 records, with 80% used for training and 20% used for validation. We keep all 2024 CVEs separate for testing, giving us a test set of 36,205 records.

Preprocessing. Before training, we cleaned the NVD records to keep only CVEs with enough usable information. We removed entries that were missing scoring analysis, had descriptions shorter than 20 characters, or were not written in English. After this filtering step, about 60% of the raw CVEs remained for each year.

For each remaining record, we extracted the fields needed for our models: the CVE ID, description, base score, full scoring string, severity label, CWE category, and the eight score components. Finally, we saved the processed data as Python pickle files, which preserve data types better than CSV files and avoid formatting issues with special characters.

Data example. Figure 1 shows a representative record from the 2024 test set. The raw description is the only model input; all structured fields on the right are predicted outputs.

Input — CVE-2024-0003:

“A condition exists in FlashArray Purity whereby a malicious user could use a remote administrative service to create an account on the array allowing privileged access.”

Target output (JSON):

```
{
  "cvss_score": 7.2,
  "severity": "HIGH",
  "cwe": "CWE-269",
  "attack_vector": "NETWORK",
  "attack_complexity": "LOW",
  "privileges_required": "HIGH",
  "user_interaction": "NONE",
  "scope": "UNCHANGED",
  "confidentiality": "HIGH",
  "integrity": "HIGH",
  "availability": "HIGH"
}
```

Figure 1: Example dataset record. The CVE description (left) is the sole model input; the structured JSON (right) is the prediction target.

Class distribution. The dataset is imbalanced. For severity, about 13% of CVEs are critical, 36% are high, 49% are medium, and only 2% are low. This means the model sees many more medium and high examples than low examples.

The CWE labels are also uneven. Common weaknesses like CWE-79, which is cross-site scripting, and CWE-89, which is SQL injection, appear thousands of times. Many other CWE categories appear fewer than 50 times. Because of this imbalance, the model has a harder time learning rare labels, especially low-severity CVEs and uncommon CWE types.

Features. For the classical baseline models, we represent each CVE description using TF-IDF bigrams. This means the model looks at important one-word and two-word phrases in the text. We make all text lowercase, remove common English stop words, keep only terms that appear in at least two documents, and limit the vocabulary to 50,000 terms.

For LoRA fine-tuning, we use the raw CVE description as the input. Each example is formatted as a conversation-style JSONL record, where the model receives the vulnerability description, and the expected response is the target enrichment output in JSON format.

4 Methods

We treat CVE enrichment as supervised multi-task learning: one text input mapped to four simultaneous outputs. The input is a raw CVE description, and the model predicts a CVSS base score (regression from 0.0 to 10.0), a severity label (four-class classification over CRITICAL, HIGH, MEDIUM, and LOW), a CWE category (multi-class classification across roughly 150 weakness categories), and eight individual CVSS vector components (attack vector, attack complexity, privileges required, user interaction, and others), each treated as a separate categorical prediction.

TF-IDF + Logistic Regression. TF-IDF converts each CVE description into a sparse vector by upweighting tokens that are distinctive to a document relative to the corpus, so rare but informative terms like *heap overflow* receive more weight than common words like *the*. Logistic regression then learns a linear decision boundary over this token space. For the K -class severity task, the model computes a raw score $\mathbf{w}_k^\top \mathbf{x}$ for each class and converts these into a probability distribution via softmax:

$$P(y = k | \mathbf{x}) = \frac{e^{\mathbf{w}_k^\top \mathbf{x}}}{\sum_{j=1}^K e^{\mathbf{w}_j^\top \mathbf{x}}}$$

The class with the highest probability is selected as the prediction. Parameters are learned by minimizing negative log-likelihood with L2 regularization ($C=1.0$, 1000 iterations), which penalizes large weights to prevent overfitting in the high-dimensional TF-IDF feature space. CVSS score regression uses Ridge regression ($\alpha=1.0$), which adds the same L2 penalty to the least-squares objective. These baselines are fast, interpretable (weights correspond directly to tokens), and establish what surface-level keyword patterns alone can achieve. Their core limitation is that TF-IDF discards word order and treats descriptions as bags of tokens, so they cannot capture negation, qualifying context, or multi-word semantic relationships.

TF-IDF + Random Forest. Random forests build an ensemble of T decision trees ($T=200$, max depth 30), each trained on a bootstrap sample of the data with a random subset of features considered at each split. Unlike logistic regression, each tree learns axis-aligned splits over subsets of the TF-IDF feature space, allowing the ensemble to capture non-linear co-occurrence patterns between tokens. The final prediction aggregates across all trees by majority vote for classification and by averaging for regression. We include this model to test whether non-linearity over TF-IDF features yields meaningful gains over logistic regression; if not, it confirms that the bottleneck is the bag-of-words representation rather than the classifier.

Fine-Tuned Phi-3.5-mini with LoRA. Unlike TF-IDF models, a transformer builds a dense, context-sensitive representation of each token by attending over the full input sequence. For each token, the model learns query, key, and value projections (Q , K , V) and

computes a weighted sum over all other tokens:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

The $\sqrt{d_k}$ term prevents dot products from growing too large in magnitude as dimension increases. Multiple attention heads run in parallel and are concatenated, allowing the model to jointly attend to different aspects of the description simultaneously, such as the attack mechanism and the required privilege level.

Phi-3.5-mini is a 3.8B-parameter autoregressive transformer decoder from Microsoft, chosen for its strong reasoning capability at small scale and its ability to fit within a 16GB device memory budget. Full fine-tuning of all 3.8B parameters is infeasible on consumer hardware. We instead apply Low-Rank Adaptation (LoRA) (9), which freezes the pretrained weights and injects trainable low-rank perturbations into selected layers. Concretely, for a frozen weight matrix $W \in \mathbb{R}^{d \times k}$, LoRA introduces two small matrices $A \in \mathbb{R}^{d \times r}$ and $B \in \mathbb{R}^{r \times k}$ with rank $r \ll \min(d, k)$, so the effective weight during fine-tuning becomes:

$$W_{\text{eff}} = W + AB$$

Only A and B are updated; the original W is never modified. At rank $r=4$ applied to 4 transformer layers, this yields approximately 5M trainable parameters out of 3.8B (0.13%), dramatically reducing both memory and compute. We trained using Apple’s MLX framework on Apple Silicon with batch size 1, learning rate 1×10^{-4} , and 300 iterations; training loss decreased from approximately 3.5 to 0.78, and peak memory usage was ~ 9 GB.

To produce all four enrichment outputs in a single forward pass, we frame each training example as a chat-style exchange: the CVE description is the user message, and a structured JSON object containing all target fields is the assistant response. The model learns to generate this JSON autoregressively using cross-entropy loss, which penalizes the model when generated tokens differ from the target. At inference time, we use greedy decoding with temperature 0 for deterministic output, capping generation at 300 tokens. The model produced well-formed JSON in over 99% of test cases, enabling automatic parsing and evaluation.

Lower-Bound Probe. To establish the minimum model scale at which LoRA fine-tuning remains viable, we fine-tuned Qwen 2.5 0.5B under the identical LoRA configuration and training procedure. This model performed substantially worse than Phi-3.5-mini, confirming that a minimum parameter threshold exists below which the model lacks the representational capacity needed for multi-output CVE enrichment. We include it as a lower-bound comparison rather than a deployment candidate.

5 Experiments, Results, and Discussion

Hyperparameter Selection. For logistic regression we used $C=1.0$ and 1,000 iterations; for random forests, 200 trees at max depth 30 (accuracy plateaued past 200 trees). For LoRA we ran a small grid over iterations $\{300, 1,000\}$ and adapted layers $\{4, 8\}$ on a held-out validation split of 2022–2023 data. The 300-iteration, 4-layer configuration performed best: 1,000-iteration variants overfit (training loss 0.59, severity accuracy 33% vs. 58% at 300 iterations), and 8 layers gave marginal improvement at higher memory cost. Learning rate was 10^{-4} (9). We used a strict temporal split (2022–2023 train, 2024 test) rather than k -fold to prevent leakage of future CVE patterns.

Evaluation Metrics. We report three metrics. **Classification accuracy** for severity and CWE: $\text{Accuracy} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\hat{y}_i = y_i]$.

Mean Absolute Error for CVSS score regression: $\text{MAE} = \frac{1}{N} \sum_{i=1}^N |\hat{s}_i - s_i|$, $s_i \in [0, 10]$. **Partial-match accuracy** for the CVSS vector, which reports the mean fraction of the 8 components correct per example rather than requiring an exact full-string match: $\text{Partial-match} = \frac{1}{N} \sum_{i=1}^N \frac{1}{8} \sum_{c=1}^8 \mathbf{1}[\hat{v}_{i,c} = v_{i,c}]$. Full-string exact match is near-zero for all models and therefore uninformative.

Quantitative Results. Table 1 reports performance on held-out 2024 CVEs. Classical models are evaluated on the full 36,205-example test set. LoRA results are from a curated 12-example sample (marked [†]). Zero-shot results for large open-source models are evaluated on a 500-example random sample from the same test set (marked [‡]).

Model	n	Severity	CWE	CVSS MAE
TF-IDF + LR	36,205	67.3%	65.1%	0.974
TF-IDF + RF	36,205	66.9%	65.6%	0.974
Qwen-2.5-72B (zero-shot)	500 [‡]	61.1%	62.1%	1.121
Llama-3.3-70B (zero-shot)	500 [‡]	49.3%	57.1%	1.508
Phi-3.5 LoRA (300 iter.)	12 [†]	58.3%	50.0%	1.9
Phi-3.5 LoRA (1000 iter.)	12 [†]	33.3%	—	—
Qwen 2.5 0.5B LoRA	12 [†]	0.0%	0.0%	—

Table 1: Summary results. [†]LoRA on 12-example curated sample (directional only). [‡]Zero-shot LLMs on 500-example random sample. CVSS vector partial-match not computed for LoRA models; zero-shot LLMs achieved 78.2% (Qwen-2.5-72B) and 76.6% (Llama-3.3-70B).

LR and RF are essentially tied (severity within 0.4%, MAE identical), confirming that TF-IDF bigrams already produce an approximately linear decision boundary, so non-linear tree capacity adds nothing (2). The zero-shot LLM results are striking: despite having orders of magnitude more parameters, both Qwen-2.5-72B (61.1% severity, 62.1% CWE) and Llama-3.3-70B (49.3%, 57.1%) fall below the TF-IDF + LR baseline, consistent with prior findings that zero-shot LLMs underperform statistical models on structured vulnerability prioritization (8). TF-IDF captures domain-specific keyword patterns, such as “heap overflow,” “unauthenticated,” “adjacent network”, that zero-shot prompting cannot reliably activate without fine-tuning. Qwen-2.5-72B comes closest (61.1% vs. 67.3%) but still lags on severity, and both models achieve lower CVSS vector partial-match (76.6–78.2%) than the classical baselines (86.2–86.8%). On the 12-example sample, Phi-3.5 at 300 iterations outperforms classical models on CWE (50% vs. 31–35%) while underperforming both

zero-shot LLMs on CWE (50% vs. 57–62%), suggesting that even minimal fine-tuning helps over keyword baselines but cannot yet match the semantic breadth of a 70B-parameter model on this task.

The confusion matrix (Figure 2) shows CRITICAL–HIGH mutual confusion dominates (2,050 and 1,492 errors respectively), while LOW recall collapses to 0.6% due to severe class imbalance (2% of training data). Table 2 confirms this per-tier: MEDIUM (F1 77.2%) is inflated by its majority share (49% of training), while CRITICAL (F1 50.6%) suffers because it shares vocabulary with HIGH (“remote,” “unauthenticated”) and is distinguished only by scope nuances TF-IDF cannot capture. LOW is the most extreme failure: 0.6% recall despite 36.4% precision, meaning the model almost never predicts LOW even when it would be correct.

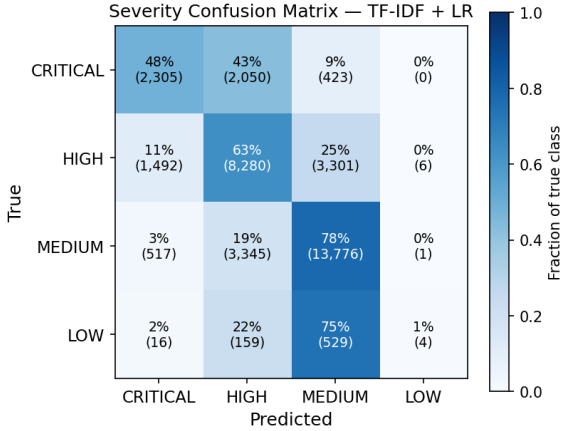


Figure 2: Severity confusion matrix (TF-IDF + LR, 36,205 test CVEs). CRITICAL–HIGH confusion dominates, LOW recall is near zero.

Tier	<i>n</i>	Precision	Recall	F1	CWE Acc.	MAE
CRITICAL	4,778	53.2%	48.2%	50.6%	64.1%	1.23
HIGH	13,079	59.9%	63.3%	61.5%	58.3%	0.80
MEDIUM	17,639	76.4%	78.1%	77.2%	71.2%	0.96
LOW	708	36.4%	0.6%	1.1%	44.8%	2.90

Table 2: TF-IDF + LR breakdown by true severity tier (36,205 test CVEs). Precision, recall, and F1 are per-class severity metrics; CWE accuracy and MAE are conditioned on the true tier.

Figure 3 and Table 3 break down per-component CVSS vector accuracy for the classical models. The 86.8% partial-match average (LR) contrasts with near-zero full-string exact match. Components stated explicitly in descriptions (Attack Complexity, Scope) cluster above 91%; Privileges Required (rarely mentioned directly) is hardest at 76.9%. Per-component CVSS vector accuracy was not computed for LoRA models due to the limited 12-example evaluation sample.

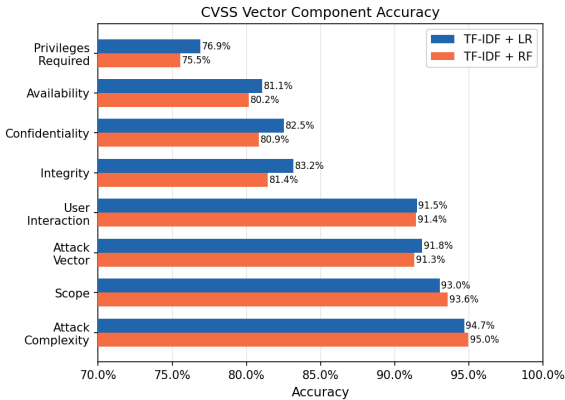


Figure 3: Per-component CVSS vector accuracy (LR vs. RF). Lexically explicit components cluster above 91%; implicit components fall below 84%.

Component	LR	RF
Attack Complexity	94.7%	95.0%
Scope	93.0%	93.6%
Attack Vector	91.8%	91.3%
User Interaction	91.5%	91.4%
Integrity	83.2%	81.4%
Confidentiality	82.5%	80.9%
Availability	81.1%	80.2%
Privileges Required	76.9%	75.5%
<i>Partial match</i>	<i>86.8%</i>	<i>86.2%</i>

Table 3: Per-component CVSS vector accuracy (36,205 test CVEs).

By CWE category (Figure 4), syntactically distinctive classes are nearly solved: CWE-79 (XSS, 98.4%) and CWE-89 (SQL injection, 98.9%) appear verbatim in descriptions. Abstract categories remain hard regardless of frequency: CWE-863 (24%), CWE-77 (38%), CWE-94 (41%) require semantic inference that bag-of-words cannot provide.

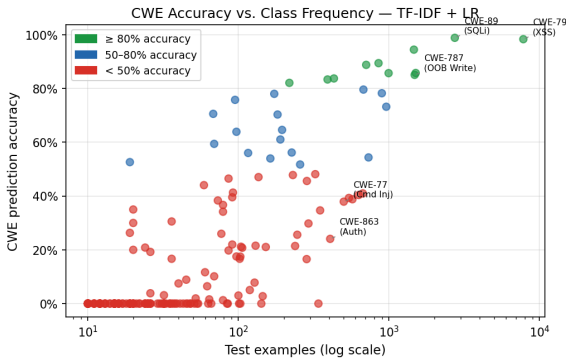


Figure 4: CWE accuracy vs. class frequency (log scale). Syntactically distinctive CWEs cluster top-right; abstract authorization/injection categories fall bottom-left regardless of frequency.

Qualitative Analysis (Phi-3.5 LoRA). Figure 5 shows Phi-3.5 collapsed to two outputs only: CVSS 9.8 (CRITICAL) and 5.4 (MEDIUM), never predicting HIGH or LOW. Of 3 actual HIGH cases, 2 were predicted CRITICAL and 1 MEDIUM; both LOW cases were predicted MEDIUM. This is a direct consequence of class imbalance interacting with limited LoRA capacity (0.13% of parameters): the model minimizes training loss by predicting dominant modes. No oversampling or class-weighted loss was applied during training; addressing this imbalance is the primary mitigation identified for future work.

Two representative failures illustrate the underlying bias. A local privilege-escalation CVE (“local authenticated user, low privileges”) was predicted CVSS 9.8, NETWORK because most CRITICAL training examples involve remote code execution. A physical USB attack (“physical access,” “USB packets”) was also predicted NETWORK; the learned prior overrode explicit lexical evidence. Conversely, two successes show what the model can do when priors align: remote unauthenticated SQL injection was correctly predicted CVSS 9.8, CRITICAL, CWE-89, NETWORK; stored XSS was correctly predicted MEDIUM, CWE-79, user interaction required.

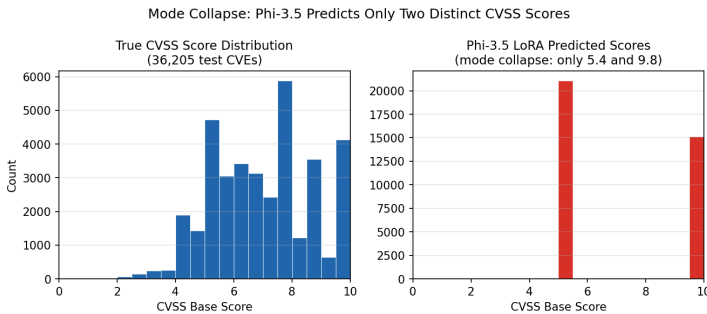


Figure 5: True CVSS distribution vs. Phi-3.5 LoRA predictions. Model collapses to 5.4 and 9.8, losing the continuous spread of the true distribution.

Discussion. The LR/RF tie confirms that the performance ceiling for classical models is set by the bag-of-words representation, not the classifier. The zero-shot LLM results reinforce this: even a 72B-parameter model cannot match TF-IDF on severity without task-specific fine-tuning, confirming that CVE descriptions contain domain-specific lexical signals that general-purpose instruction-following does not reliably exploit. Mode collapse is the primary failure mode for LoRA: the 1,000-iteration model overfit (lower training loss, worse test accuracy), mitigated here by early stopping at 300 iterations; validation-based early stopping and class-weighted loss would be the rigorous fixes. The partial-match metric reveals that CVSS vector prediction is substantially stronger than exact match suggests: components with explicit lexical signals are near-solved, while those requiring implicit inference remain the hardest targets.

6 Conclusion and Future Work

This project shows that small language models can perform meaningful CVE enrichment on consumer hardware. TF-IDF classical baselines achieve 67% severity accuracy, 0.97 CVSS MAE, and 87% partial CVSS vector match on 36,205 held-out 2024 CVEs. LoRA fine-tuned Phi-3.5-mini shows clear gains on CWE classification but mode-collapses on severity due to class imbalance. Qwen 0.5B fails entirely, setting ~1B parameters as the practical floor. More training is not always better: the 1,000-iteration model overfit despite lower training loss.

The most impactful next step is oversampling LOW and CRITICAL to address class imbalance, then evaluating on the full 2024 test set with per-class precision and recall. We would also compare Llama 3.2 and Gemma 2 across sizes, and benchmark against CIRCL RoBERTa (4) as a published baseline. Adversarial robustness testing is also a priority: can descriptions be perturbed to trick the model into under-rating a CRITICAL vulnerability? This would be critical to evaluate before any real deployment.

References

- [1] National Institute of Standards and Technology (NIST). (2026). Notice on National Vulnerability Database Enrichment Process Changes. <https://www.nist.gov/news-events/news/2026/04/nist-nvd-enrichment-update>
- [2] Aota, M., Aoki, S., and Ban, T. (2025). On Using Large Language Models for Vulnerability Classification. *Proceedings of the ACM KDD Conference*, pp. 3245–3256.
- [3] Shahid, M. R. and Debar, H. (2021). CVSS-BERT: Explainable NLP to Determine the Severity of a Computer Security Vulnerability from its Description. *Proceedings of the IEEE ICMLA*, pp. 1600–1607. arXiv:2111.08510.
- [4] Computer Incident Response Center Luxembourg (CIRCL). (2025). Vulnerability Severity Classification with RoBERTa. Hugging Face Model Card. <https://huggingface.co/CIRCL/vulnerability-severity-classification-roberta-base>
- [5] Anonymous. (2026). Fine-Tuned RoBERTa for CVE-to-CWE Classification: Matching Large Model Performance with Lightweight Architectures. *arXiv preprint arXiv:2603.14911*.
- [6] Levantesi, A., Demirsoy, A., and Aksu, F. (2025). CVE-LLM: Automated Vulnerability Assessment Using Fine-Tuned Large Language Models. *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, pp. 12450–12459. arXiv:2502.15932.
- [7] Mukherjee, A., Patel, R., and Kumar, S. (2025). CASEY: Automating Bug Report Categorization Using Fine-Tuned Large Language Models. *arXiv preprint arXiv:2501.18908*.
- [8] Jacobs, J., Romanosky, S., Roytman, M., and Adjerid, I. (2025). Benchmarking LLMs on Vulnerability Prioritization. Empirical Security Research Report, FIRST.org. <https://research.empiricalsecurity.com/research/benchmarking-llms>
- [9] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. (2021). LoRA: Low-Rank Adaptation of Large Language Models. *arXiv preprint arXiv:2106.09685*.
- [10] Databricks Security Engineering. (2024). VulnWatch: AI-Powered Vulnerability Prioritization for Software Libraries. Databricks Engineering Blog. <https://www.databricks.com/blog/vulnwatch-vulnerability-prioritization>
- [11] Fraunhofer FKIE. (2024). NVD JSON Data Feeds. <https://github.com/fkie-cad/nvd-json-data-feeds>.